

Sistema Inteligente con STM32 para Conducción de un Tractor

Jonathan Jurgen Zabala Peña¹, Omar Andres Garcia Rivera²,
Luis Gustavo Cons Gastélum Rocha³ and Ángel Enrique Montes Pacheco⁴

I. INTRODUCCIÓN

A. Contexto general

En la agricultura industrial actual se utilizan una amplia gama de equipos y productos para eficientar el proceso de: cultivo, crecimiento, y recolección. Es en este rubro donde máquinas como los tractores y cosechadoras brillan por su constante mejora, mejoras que se ven reflejadas en el aumento de la eficiencia de la cosecha y la recolección. Pero la mejora de estas bestias mecánicas no ocurre de la noche a la mañana, es un proceso que toma tiempo y dinero, y es multifactorial. No es un solo apartado el que se mejora, hay muchos, y cada vez que se mejore uno, se le debe hacer su respectiva prueba, la cual puede llegar a tomar demasiado tiempo para una industria que exige velocidad.

El testeo de software y chips es particularmente demandante en tiempo debido a que se necesitan grandes cantidades de datos reales para probar todas las posibilidades dentro de estos programas. Datos que toman demasiado tiempo en recolectarse. Es en este apartado donde las simulaciones virtuales nos ayudan a generar estos datos.

*This work was not supported by any organization

¹H. Kwakernaak is with Faculty of Electrical Engineering, Mathematics and Computer Science, University of Twente, 7500 AE Enschede, The Netherlands h.kwakernaak at papercept.net

²P. Misra is with the Department of Electrical Engineering, Wright State University, Dayton, OH 45435, USA p.misra at ieee.org

B. Delimitación del objeto de estudio

Para lo que compete a esta investigación y su desarrollo, esta se enfocará en las herramientas y/o vehículos usados en la agricultura, específicamente aquellos que contribuyen al arado del suelo, como lo es el tractor, vehículo que será nuestro objeto de estudio, así como base para el desarrollo de nuestro prototipo.

El tractor presenta múltiples funciones dentro del área donde es usado, en este caso, la agricultura. El tractor desempeña trabajos no solo en el sembrado, si no también en labores de carga, siembra, cosecha y fertilización, es por esto que es necesario también definir la funcionalidad en la que estaremos enfocados. Por motivos de nuestra investigación y desarrollo de un prototipado, el enfoque que se tomará, es en el uso de carga del tractor.

C. Planteamiento del problema

Debido a su uso como maquinaria de arrastre o carga y al entorno en el que se trabaja, estos vehículos pueden presentar una menor eficiencia, presentando averías de forma común en la potencia de la transmisión, sobrecalentamiento del motor, o averías en su sistema hidráulico o eléctrico [1], aunque también esta baja eficiencia puede depender del sistema de labranza o la carga puesta sobre el tractor, ejemplo de esto son los suelos intensivamente labrados, donde los tractores presentan su menor eficiencia [2].

Cabe mencionar que al ser maquinaria se deben tener consciencia de los distintos peligros que se

tienen al momento de trabajar, siendo los mas comunes: aplastamiento por el vuelco del tractor, arrollamiento, arrastre, o proyección de partículas fragmentos o objetos [3]. Con esto en mente, se busca la reducción de los riesgos al trabajar con esta maquinaria.

Es por ello que necesitamos de un sistema que pueda, a partir de un modelo matemático, generar de forma aleatoria cientos o miles de datos referentes a: velocidad angular, radio de la rueda y relación de la transmisión de un tractor con la finalidad de calcular las revoluciones por minuto del mismo.

Además de la inclusión de sensores y mecanismos que nos permitan controlar y visualizar de forma remota el tractor. Esto con el fin de simular de forma segura su comportamiento, así podremos estimar su eficiencia de acuerdo a su carga de trabajo y el peligro del mismo.

D. Justificación

Las tractores presentan comúnmente fallas en la potencia y velocidad debido a la acumulación de desgaste en sus mecanismos interno debido al esfuerzo que realizan en los campos, dicho desgaste suele acrecentar, requiriendo de equipo y material especializado para su pronta reparación.

Estos problemas suelen representar pérdidas significativas a los agricultores, dada la inversión que se realiza en las refacciones, equipo y sueldos; sin contar un retraso en el avance de su trabajo. Por ello es necesario realizar el análisis anteriormente descrito, para conocer el comportamiento del tractor y poder construir un prototipo escalable.

E. Marco teórico

En el mundo ya se han creado programas que permiten generar datos aleatorios que simulan el comportamiento de algo siguiendo una fórmula o pauta específica. Está demás mencionar ejemplos ya que usamos bastante estos programas en nuestro día a día sin darnos cuenta; como los

generadores de contraseñas aleatorias, lo cuales no son aleatorios del todo y tienen que seguir alguna fórmula ya que las máquinas realmente no pueden generar aleatoriedad de forma natural.

1) Conceptos relevantes: Sistemas embebidos

Un Sistema en Chip (SoC) es un circuito integrado que contiene los componentes principales de un sistema informático dentro de una misma pieza, incluyendo cosas como CPU, RAM, memoria Flash, GPU, periféricos de entrada y salida, entre otros. Estos sistemas tiene como objetivo eficientar el consumo eléctrico y costos de producción de los dispositivos electrónicos, además de reducir el espacio físico que los componentes ocupan [12]. Dentro de los SoC, se destacan los microcontroladores, un sistema cuyo objetivo principal es controlar sensores y actuadores. Estos generalmente se componen de un procesador (de 8 a 32 bits), una memoria flash de programa, una memoria de datos (SRAM) y otros módulos de control. Existen diversos tipos de microcontroladores como los AVR, STM32 o ESP32. Estos se caracterizan por ser de bajo costo, requerimientos de alimentación y facilidad de programación [9].

Los SoC y los microcontroladores son esenciales para la innovación y creación de nuevas tecnologías, ya que permiten realizar productos con un número reducido de componentes fuera del chip. Simplificando cosas como PCBs, tiempos y complejidad de prototipos y, en la mayoría de casos, entornos de desarrollo amigables y con mucha documentación. Además, debido a su accesibilidad general, permiten que casi cualquier persona pueda desarrollar sus propias ideas y proyectos, abriendo así la posibilidad de que surjan nuevas aplicaciones tecnológicas en diversos rubros y bajo diferentes filosofías de trabajo [8].

- Aplicaciones de sistemas embebidos
 - Dispositivos médicos (marcapasos, monitores de glucosa)
 - Sistemas de control automotriz (ECU, ABS, ADAS)

- Domótica e IoT (termostatos, sensores inteligentes)
- Equipo industrial (PLC, robots, máquinas CNC)
- Electrónica de consumo (televisores, cafeteras, lavadoras)

Ventajas	Desventajas
• Bajo consumo energético • Bajos costos productivos y de desarrollo • Tamaño reducido • Documentación de uso amplia • Múltiple hardware integrado	• Recursos CPU, RAM y almacenamiento limitados • Dependencia de fabricantes (soporte, disponibilidad, etc.) • Menor flexibilidad a cambios en funciones

TABLE I: Ventajas y desventajas de los sistemas embebidos

Sistemas operativos en tiempo real para sistemas embebidos

Un Sistema Operativo en Tiempo Real/Real Time Operating System (RTOS) es un sistema operativo diseñado para gestionar recursos de hardware en sistemas embebidos con restricciones temporales estrictas. Su principal objetivo es que las diferentes tareas cumplan con sus deadlines específicos, evitando que se pierda información o funciones fundamentales, independiente de la carga o estado del sistema [11].

Los RTOS ofrecen primitivas de planificación (scheduling), gestión de tareas, sincronización (semáforos, mutexes), comunicación entre procesos (colas de mensajes) y temporizadores de alta resolución. Estos generalmente son utilizados mediante librerías y sistemas estandarizados como FreeRTOS, Zephyr RTOS, VxWorks, QNX y microC/OS [7].

- Aplicaciones de sistemas operativos en tiempo real para sistemas embebidos
 - Sistemas de control de vuelo en aviones y drones.

- Equipos médicos de monitoreo en tiempo real (ventiladores, desfibriladores).
- Sistemas de frenado antibloqueo (ABS) y bolsas de aire en automóviles.
- Robots industriales con control de movimiento preciso.
- Estaciones base de telecomunicaciones y equipos de red.

Ventajas	Desventajas
• Gestión estructurada de múltiples tareas concurrentes mediante planificación por prioridades. • Mayor modularidad y mantenibilidad del código al separar tareas independientes. • Primitivas de sincronización (mutexes, semáforos) que simplifican el acceso a recursos compartidos. • Facilita el escalado del sistema: agregar nuevas funcionalidades sin reestructurar el superloop. • Mejor integración con pilas de protocolos (TCP/IP, Bluetooth, USB) diseñadas para RTOS.	• Overhead adicional de memoria (RAM y flash) por el kernel del RTOS. • Latencia de contexto por el cambio de tareas (context switching) • Mayor complejidad de diseño: es necesario gestionar prioridades, deadlocks e inversiones de prioridad.

TABLE II: Ventajas y desventajas de los sistemas operativos en tiempo real

Linux Embebido

Linux Embebido se refiere al uso del kernel Linux en sistemas embebidos de mayor capacidad, como procesadores ARM Cortex-A, MIPS o x86 con recursos suficientes (típicamente desde 32 MB de RAM y 64 MB de almacenamiento). Linux provee un sistema operativo completo con gestión de procesos, sistema de archivos, pila de red completa, controladores de dispositivo y soporte

para aplicaciones de usuario [13].

- Aplicaciones de Linux
 - Routers y gateways de red (OpenWRT).
 - Cámaras IP y sistemas de videovigilancia.
 - Terminales de punto de venta (POS) y cajeros automáticos.
 - Sistemas de información y entretenimiento vehicular (IVI).
 - Dispositivos IoT de gama alta (pasarelas, edge computing).
 - Equipos industriales con HMI (Human-Machine Interface).

Ventajas	Desventajas
• Ecosistema maduro con amplia disponibilidad de controladores de hardware. • Soporte para lenguajes y frameworks modernos (Python, Node.js, contenedores Docker). • Gran comunidad y abundante documentación. • Capacidad para ejecutar aplicaciones complejas y multitarea avanzada. • Actualizaciones de seguridad y parches constantes.	• Requiere mayor cantidad de recursos (RAM, flash). • Tiempos de arranque más largos. • El kernel Linux estándar no es de tiempo real. • Mayor superficie de ataque en ciberseguridad respecto a firmware minimalista.

TABLE III: Ventajas y desventajas de Linux

Bash Scripts y Python en Linux Embebido

Bash (Bourne Again SHell) es el intérprete de comandos estándar en la mayoría de distribuciones Linux. En sistemas embebidos, los scripts de Bash se utilizan para automatizar tareas del sistema: inicialización de periféricos, monitoreo del estado del hardware (temperatura, uso de CPU), gestión de logs, actualizaciones OTA (Over-The-Air) y orquestación de servicios. Su integración nativa con el shell de Linux permite interactuar directamente

con el sistema de archivos, variables de entorno y herramientas del sistema sin necesidad de compilación [13].

Python es uno de los lenguajes más utilizados en Linux embebido gracias a su simplicidad y su vasto ecosistema de bibliotecas. En el contexto embebido, Python se emplea para:

- Adquisición y procesamiento de datos de sensores (bibliotecas como RPi.GPIO, smbus2, pyserial).
- Comunicación con servicios en la nube (MQTT, HTTP REST APIs) mediante paho-mqtt o requests.
- Interfaces gráficas ligeras con tkinter o PyQt.
- Análisis de datos en el edge con NumPy, SciPy o TensorFlow Lite.
- Automatización de pruebas y despliegue de aplicaciones.

La combinación de Bash para administración del sistema y Python para lógica de aplicación representa un enfoque práctico y productivo en el desarrollo de productos con Linux Embebido [13].

2) *Revisión de literatura:* Para esta investigación, se hizo la revisión de investigaciones referentes a problemas comunes en el área agrícola, abarcando desde accidentes por parte de las personas, hasta fallas y riesgos de la maquinaria. Del mismo modo, también se hicieron consultas a artículos que presentarán información acerca del estado del tractor o su rendimiento, al momento de realizar actividades pesadas como la labranza.

En estos artículos se señalan riesgos comunes de la maquinaria agrícola y su rendimiento en labores de labranza. Estos artículos fueron seleccionados debido al contenido que presentan, dado que resulta beneficioso para la investigación y el futuro desarrollo de nuestro prototipo, el conocer las cosas por mejorar en el diseño de los tractores.

F. Objetivos

Objetivo general

- Implementar un sistema inteligente de conducción de un tractor a escala mediante la integración de sistemas de Hardware distribuidos para simular movimientos y control en un entorno específico.

Objetivos específicos

- Realizar un entorno de simulación de parámetros del tractor para proyectar valores relevantes como RPM.
- Programar un microcontrolador STM32 para gestionar el control, movimiento y funcionamiento del tractor.
- Desarrollar un dashboard de visualización de datos en tiempo real con Raspberry Pi.

En cuanto a requerimientos, el sistema debe integrar un prototipo físico de vehículo construido con materiales accesibles que sea capaz de ejecutar un modelo de motor-transmisión en tiempo real. El control se divide en dos nodos principales: un microcontrolador STM32 operando con RTOS para el procesamiento de señales y cálculo de modelos, y una Raspberry Pi que gestiona la interfaz de usuario inalámbrica para el mando de dirección, frenado y aceleración. Finalmente, el sistema debe visualizar gráficamente variables como RPM.

G. Hipótesis

La implementación de un sistema de conducción distribuido, que separa las tareas de procesamiento crítico de las de interfaz de usuario, permitirá un control eficiente y seguro del tractor en un entorno agrícola. Se espera que cada sección del proyecto se realice de manera rápida y eficiente, evitando errores o retardos críticos. El tractor será capaz de cumplir con las tareas y especificaciones designadas en el reto sin problemas o dificultades relevantes.

II. PROPUESTA

A. Metodología

Para la realización de este proyecto se tuvo que investigar seriamente las distintas problemáticas

presentas en la industria agrícola, referente a una de sus maquinarias; el tractor. Tomando como referencia la problemática presentada por nuestro socio formador, finalmente pudimos dar con una propuesta viable y escalable para la problemática definida con anterioridad

Posterior a la investigación y delimitación de nuestra problemática, tuvimos que instruirnos primeramente, para ser capaces de hacer uso de forma adecuada, las diferentes placas y software que se tendría que usar en la elaboración del presente proyecto.

Para la elaboración del presente proyecto, y para la instrucción que estábamos recibiendo, se nos presentaron las herramientas que deberíamos usar para nuestro proyecto: STM32, ESP32 Y una Raspberry Pi4. Además se nos instruyó en diferentes tipos de software y arquitecturas como: Linux (comandos básicos), Jerarquía en la computadora, Kernel, Assembly, LEGv8, FreeRTOS, STM32 CubeMX y Thonny, cada uno de estos fue fundamental para la elaboración del prototipo final.

Siguiendo lo anterior, fue necesario revisar la lista de materiales sugerida por nuestro socio formador, así como considerar materiales alternativos para nuestro prototipo, finalmente se optaron por materiales resistentes y ligeros para la carcasa, jumpers para las conexiones internas del tractor y componentes físicos y electrónicos para la funcionalidad del tractor; además de las placas anteriormente mencionadas. Con esto en mente se formó una lista de materiales necesarios para dar inicio a la producción del proyecto, aunque se tuvieron que comprar más durante el camino debido a imprevistos.

Con el conocimiento de lo que se debía realizar, así como de los materiales necesarios para la construcción del prototipo, se formaron sub-equipos que permitieran un trabajo paralelo, de esta forma el trabajo no se quedaría atascado en una sola tarea, y podríamos abarcar más, solamente reuniendo estos sub-equipos para hablar sobre avances y la organización del trabajo final.

Finalmente, se realizó la configuración de

las placas, y se crearon los códigos necesarios para el funcionamiento del sistema, partiendo de pequeños avances que nos permitiesen probar parte del sistema y verificar su funcionamiento. El primero de estos avance realizados buscó simular el almacenamiento y envío de los posibles datos de un tractor en el programa de InfluxDB, para posteriormente graficarlos en el ambiente de la Raspberry Pi; esto se logró mediante la codificación en el ambiente de Grafana, dentro de la Raspberry Pi (véase en [Anexo A](#) (pág. 12). Como segundo avance se decidió probar el envío de datos reales entre las distintas placas, para ello se re-configuraron las placas para el envío de datos mediante distintos protocolos (protocolo UART, MQTT y Wifi), y se añadieron programas que permitiesen el guardado de la información recibida, así como su graficación en tiempo real en el ambiente de la Raspberry Pi (véase Anexo A).

B. Técnicas y herramientas de ingeniería empleadas

1) Lenguajes de programación:

- **Python:** Usado para la codificación de los dashboard de los datos entregados por la ESP32 y obtenidos por el STM32 para su graficación.
- **C:** Este lenguaje de programación nos permite la configuración de pines, registros, y la programación de las tareas de la placa STM32.
- **C++:** Lenguaje que nos permite la programación y configuración de los pines del ESP32, el cual envía los datos recopilados por el STM32 hacía la Raspberry Pi.

2) Recursos:

- **STM32:** El STM32 es el encargado de recopilar la información telemetría del tractor, así como ejecutar el modelo proporcionado por nuestro socioformador.
 - Configuración:

Pin STM32	Función
PA0	Potenciometro - Aceleración (ADC)
PA1	Botón - Freno (pull-up)
PA6	LED3 - PWM TIM3_CH1
PA7	LED4 - PWM TIM3_CH2
PA9	UART TX → ESP8266
PA10	UART RX ← ESP8266
PB0	LED1 - PWM TIM3_CH3
PB1	LED2 - PWM TIM3_CH4
PB5	LCD - D6
PB6	LCD - D7
PB8	LCD - RS
PB10	LCD - Enable
PB12	LCD - D4
PB13	LCD - D5

Fig. 1: Diagrama de lectura del botón

En la figura [2] (vease de forma detallada en Anexo C (pág. 12) y Anexo D (pág. 12); búsquese como Figura 1) se puede observar los pines usados, así como sus conexiones hacia los componentes utilizados para le prototipo.

– Uso de periféricos:

- * **UART:** Se configuraron los pines TX Y RX para el envío de información.
- * **LCD:** Los pines PB5,6,8,10,12,13 se usarón para enviar la información a la pantalla LCD.
- * **Freno:** PA1 usado para recibir las señales del botón
- * **Potenciometro:** PA0 usado para recibir las señales del potenciometro
- * **LEDs:** Pines PA6,7 y PB0,1 fueron usados para vizualisar la velocidad obtenida del modelo.
- **ESP32:** Permite la transmisión de los datos recopilados por el STM32 a la Raspberry Pi, mediante la comunicación MQTT.
 - **Configuración:** Se configuro para recibir y enviar la información del modelo, recibe la información por medio del protocolo UART y lo envia por el protocolo MQTT hacía la Raspberry Pi
 - **Uso de periféricos:** Por le momento el ESP32 solamente es un intermediario entre las Raspberry Pi y el STM32.
- **Raspberry Pi4 :** Recibe los datos transmitidos por el ESP32 para mostrarlos a través de dashboard al usuario, pudiendo observar de

forma sencilla y en tiempo real la telemetría del tractor.

- **Configuración:** Se configuro para recibir los datos enviados por el ESP32 y desplegar los en su interfaz.
- **Uso de interfaz:** Se ajusto la interfaz de la Raspberry Pi para la visualización de los datos recibidos.

3) Protocolos de comunicación:

- **Wifi:** Wifi es una conexión inalámbrica entre dos dispositivos, usadas normalmente para redes inalámbricas locales o acceder a la internet [4]. Esta conexión es la que nos permite interactuar con la Raspberry Pi, se usa una red local para realizar la conexión wifi.
- **UART:** El protocolo UART o Universal Asynchronous Receiver-Transmisor, es un protocolo de comunicación que permite la transmisión de datos entre dos dispositivos [5], para esta investigación, entre el STM32 y el ESP32, para lograrlo se requiere la configuración de los pines destinados para este tipo de comunicación, los pines RX y TX de ambas placas.
- **MQTT:** Protocolo de mensajería estándar desarrollado por "OASIS" y usado comúnmente para Internet de las Cosas, debido a su eficiencia y ligereza, así como por su escalabilidad [6]. Este protocolo nos permite realizar de forma segura la conexión entre nuestra ESP32 y la Raspberry Pi, permitiendo el envío de la información del tractor a nuestra interfaz de la Raspberry Pi.

4) *Diagrama esquemático del prototipo:* En este apartado se mostrará el esquemático de nuestro prototipo, sin embargo, para poder representarlo a mayor detalle, se presentarán las conexiones de la STM32 con el tractor, y posteriormente con el resto de placas.

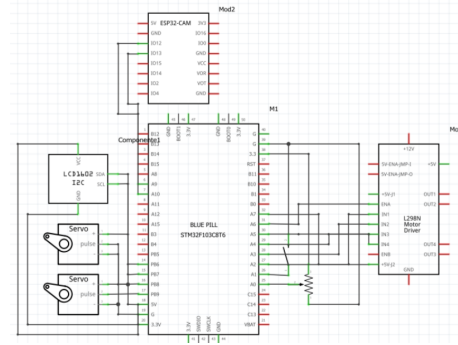


Fig. 2: Esquemático: Conexiones de la STM32 con los elementos del tractor

En la imagen anterior se observa de mejor manera la conexión física para la transmisión de la información por medio del protocolo UART. Como se observa en la figura [2] (vease de forma detallada en Anexo D (pág. 12); búsquese como Figura 2) los pines RX y TX de ambas placas se encuentran en una conexión cruzada, esto es debido a que RX (receptor) debe de ir conectado a TX (transmisor) para que pueda recibir o mandar información a la STM32.

5) *Diagramas de flujo:* En esta sección se explorarán las distintas tareas que se realizan en la placa STM32.

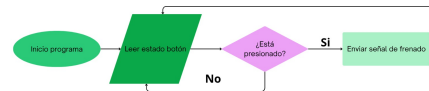


Fig. 3: Diagrama de lectura del botón

Como se muestra en la figura [3] (vease de forma detallada en Anexo D (pág. 12); búsquese como Figura 3), una de las tareas que realiza el STM32 es la continua verificación de la pulsación de un botón, el cual nos indica el freno del tractor (si se frena o no).

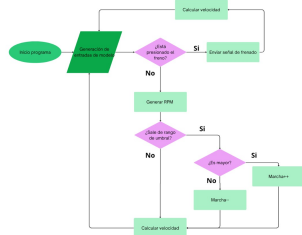


Fig. 4: Diagrama del funcionamiento de la generación de las entradas del modelo

En la figura [4] (vease de forma detallada en **Anexo D** (pág. 12); búscuese como Figura 4) podemos visualizar como es que el modelo toma las decisiones de acuerdo a diferentes condiciones, como es el calculo de la velocidad resultante si se presiona le botón de frenado, el calculo de las RPM, o el aumento de la marcha si estas revoluciones se salen del umbral especificado o su disminución en caso de que sea menor. También observamos que al final de todo el programa calculara la velocidad final, para después volver a repetir la tarea.

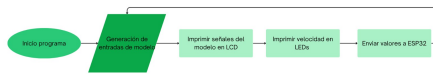


Fig. 5: Diagrama explicativo del uso del modelo

Podemos observar en la figura [5] (vease de forma detallada en **Anexo D** (pág. 12); búscuese como Figura 5), como los datos que obtuvimos del modelo [4] son utilizados por el resto de componentes. Siendo que las señales obtenidas del modelo se vizualisan en la pantalla LCD, la velocidad se ve reflejada en los LEDs, mientras que, a su vez, los datos son enviados por medio del protocolo UART a la ESP32.

Véase que después de enviar los datos resultantes, se vuelve a repetir dicha tarea, esto es lo

que nos permite simular la toma de datos en tiempo real, permitiendo observar como podría funcionar el programa cuando los datos sean tomados del entorno.

6) *Interfaz de usuario:* Para la gestión y monitoreo del sistema, se ha seleccionado el programa InfluxDB para el almacenamiento de datos, mientras que para la gráficación y creación de dashboards, se ha utilizado el programa Grafana que nos ha permitido crear la siguiente interfaz.

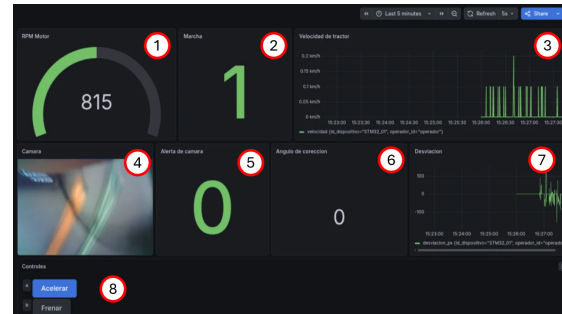


Fig. 6: Dashboards de la telemetría del tractor

Como se observa en la Fig. [6] (vease de forma detallada en **Anexo D** (pág. 12); búscuese como Figura 6) se tienen diferentes apartados que permiten una visualización más ordenada de la telemetría del tractor, a continuación se explicara cada uno.

- Apartado 1 [RPM]: En este apartado se presenta las revoluciones por minuto que tiene el tractor en ese instante de tiempo.
- Apartado 2 [Marcha]: Se muestra la marcha actual del tractor.
- Apartado 3 [Velocidad]: Muestra la velocidad del tractor en el tiempo.
- Apartado 4 [Cámara]: Visualiza los surcos por los que pasa el tractor, incorporada con un algoritmo de detección de dichos surcos.
- Apartado 5 [Alerta]: Si se detectan surcos se envía una alerta a la Raspberry Pi, la cual se visualiza en el dashboard
- Apartado 6 [Ángulo de corrección]: Muestra el ángulo de desvío del tractor.
- Apartado 7 [Desviación]: Determina la desviación respecto al surco observado en

- Apartado 8 [Controles]: Zona de comandos que permite el control del tracto a distancia.

C. Pruebas de funcionamiento del prototipo

[illegible]

Fig. 7: Representación de envío de datos a InfluxDB
- Código

- 2) *Muestra de la aceleración en el LCD:* Transmisión de los datos del modelo en la STM32 a la pantalla LCD para su visualización directa.

```
# TAREA: E디스플레이 P1_200 ms
# -----
static void task_display(void *arg)
{
    (void)arg;

    ModelState_t st = {800,0,1,2,0,0,0,0,0};
    TickType_t xLastWake = xTaskGetTickCount();

    for (;;)
    {
        vTaskDelayUntil(&xLastWake, pdMS_TO_TICKS(200));
        xQueuePeek(xQueueModel, &st, 0);

        LCD_SetCursor(1,1);
        LCD_PutStr("Hi");
        LCD_PutInt(st.act_pcel, 3);
        LCD_PutChar(' ');
        LCD_PutChar(st.mode==MODE_MANUAL?'M':'I');
        LCD_PutStr(" R ");
        LCD_PutInt(st.rpm, 4);
        LCD_PutChar(' ');

        LCD_SetCursor(2,1);
        if (st.brake) {
            LCD_PutStr("****RENO*** H:");
            LCD_PutChar((char>('0'+st.gear));
            LCD_PutChar(' ');
        } else {
            LCD_PutStr("V:");
            LCD_PutInt(st.vspeed, 3);
            LCD_PutStr("/km/h M:");
            LCD_PutChar((char>('0'+st.gear));
            LCD_PutStr(" ");
        }
    }
}
```

Fig. 8: Representación de los datos en la LCD - Código

- Representación en el código: Esto es gracias a las funciones iniciadas por LCD, las cuales envían los datos a través de los pines señalados con anterioridad (véase de forma detallada en **Anexo D** (pág. 12) y en código en **Anexo A** (pág. 12)).

3) *Comunicación con la ESP32*: Conexión entre las Raspberry Pi 4 y la ESP32 mediante el protocolo MQTT de conexión inalámbrica.

```
def on_connect(client, userdata, flags, rc, properties=None):
    if rc == 0:
        print(f"[MQTT] Conectado al broker {BROKER_IP}:{BROKER_PORT}")
        client.subscribe(TOPIC)
        print(f"[MQTT] Suscrito a '{TOPIC}'")
        _conectado.set()
    else:
        print(f"[MQTT] Error de conexion rc={rc}")
        _conectado.clear()

def on_disconnect(client, userdata, rc, properties=None):
    print(f"[MQTT] Desconectado (rc={rc})")
    _conectado.clear()
```

Fig. 9: Representación de comunicación MQTT con la ESP32

- **Representación en el código:** La función `onconnect` y `ondisconnect` permiten la conexión o des-conexión del dispositivo ESP32 (vease de forma detallada en **Anexo D** (pág. 12 y en código en **Anexo A** (pág. 12)).

D. Infraestructura

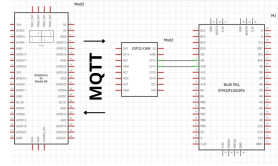


Fig. 10: Esquema de conexiones Raspberry Pi 4 + STM32 + ESP32

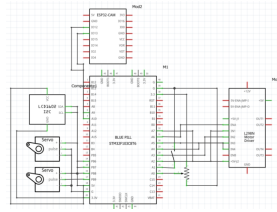


Fig. 11: Esquema de conexiones STM32 con el tractor

1) Infraestructura del sistema:

- STM32 + Raspberry Pi 4 (incluyendo ESP32): En las Figuras [10] y [11] (véase más detalladamente en **ANEXO D** (pag. 12); búsqese como Figura 7 y 8, respectivamente) se pueden visualizar las conexiones realizadas entre las distintas placas y los componentes incorporados en el tractor. A continuación se explicará a detalle estas conexiones.

- STM32 + ESP32: Conectados por medio del protocolo UART (anteriormente mencionado), la ESP32 es quien recibe los valores obtenidos de los sensores y modelos de la STM32. También es el encargado de transmitir los comandos que recibe de la Raspberry.
- ESP32 + Raspberry Pi 4: Conectados mediante el protocolo MQTT, la Raspberry recibe los datos de la ESP32 para almacenarlos en InfluxDB y mostrarlos a través de los dashboard incorporados en Grafana, finalmente, transmite comandos a la ESP32 mediante esta misma comunicación. No hay conexión física entre la ESP32 y la Raspberry Pi.

- Stm32 + Raspberry Pi 4: Comunicándose por medio de un intermediario (ESP32), la STM32 obtiene los valores que después almacena y gráfica la Raspberry Pi. La STM32 también recibe los comandos de la Raspberry, por medio de su conexión física con el intermediario.
- STM32 + Tractor: La STM32 se comunica físicamente (por medio de cables) a los distintos componentes que tiene el tractor, mediante la planificación de tareas (FreeRTOS), el sistema incorporado revisa los sensores, el puerto serial, ejecuta el modelo incorporado (dicho modelo realiza los cálculos de velocidad, aceleración y RPM) y también envía señales de acción a los actuadores incorporados que permiten el movimiento del tractor.

Como se mencionó con anterioridad, el sistema funciona mediante distintos protocolos que permiten la comunicación entre las placas, distintos algoritmos que permiten un control en tiempo real del tractor, y herramientas que permiten la visualización de la información obtenida. Como parte de este apartado, también se debe señalar las diferentes tareas que permiten el funcionamiento del sistema.

Tarea	Ejecución (ns)	Perioricidad
Tarea 1	1	10
Tarea 2	39	200
Tarea 3	32	200
Tarea 4	4	25
Tarea 5	1	10

TABLE IV: Set de tareas para el sistema

Las tareas mencionadas en [IV] hablan acerca del sistema contenido en la STM32. Para incorporar las se utilizó el sistema operativo FreeRTOS dentro de la STM32 para la creación y calendarización de estas tareas; dicha calendarización se realiza con el scheduler Rate Monotonic, el cual es un scheduler basado en prioridades, clasificado como preemption. Nuestro sistema tiene un porcentaje de uso de la CPU de un 71.5 por ciento, esto debido a como el scheduler va otorgando a cada

tarea su espacio en el procesador.

De esta forma el sistema general (Tractor + STM32 + ESP32 + Raspberry Pi 4) logra obtener los datos en tiempo real, y por consiguiente también controlar el prototipo en tiempo real.

2) *Recursos utilizados:* En este apartado declaramos todos los recursos, tanto como componentes físicos y electrónicos, como los software empleados para la construcción del prototipo.

- Recursos: Hardware y Prototipo:

Componente	Función	Conexión
L298N No. 1	Control	STM32
L298N No. 2	Control	STM32
Motor DC A	Movimiento	L298N No. 1
Motor DC B	Movimiento	L298N No. 1
Motor DC C	Movimiento	L298N No. 2
Motor DC D	Movimiento	L298N No. 2
Servomotor	Movimiento	STM32
Servomotor	1	Movimiento
Micro-servomotor	Movimiento	STM32
Pantalla LCD	Representación	Módulo LCD
Módulo LCD	Control	STM32
FTDI	Comunicación	STM32
STM32 (Blue Pill F103C8T6)	Control	ESP32
ESP32	Comunicación	STM32/Raspberry Pi 4
Raspberry Pi 4	Control y representación	ESP32

TABLE V: Componentes del sistema

- Recursos: Software

Software	Función
Solidworks	Modelación
CAD	Modelación
Visual Studio Code	Programación
Thonny	Programación
InfluxDB	Base de Datos
Grafana	Dashboards
STM32 CubeMX	Configuración
FreeRTOS	Sistema operativo

TABLE VI: Software usado para el sistema

Como se muestra en las tablas [V] y [VI] (véase con mayor detalle en **ANEXO C** (pág. 12)) de estos

componentes tanto electrónicos como físicos, de los cuales se destacan: El modulo LCD; permite un manejo más sencillo y eficiente de la pantalla LCD, el L298N o puente H; permite el control de los 4 motores. Por su parte, cada software mencionado en la tabla [VI] fue indispensable en cada parte del prototipo, ya sea en su contraparte física (Solidworks y CAD); que comprende la carcasa, neumáticos y la estructura general del prototipo, o en su sistema (VS Code, Thonny, InfluxDB, Grafana, STM32 CubeMX y FreeRTOS); creando tareas, configurando alguna de las placas, o en el diseño y almacenamiento de los datos generados.

III. RESULTADOS

1) *Evaluación e Interpretación:* Referente a los objetivos del proyecto, así como a lo presentado en este documento, se demuestra el funcionamiento del prototipo, logrando en él, la implementación de un sistema que opera en tiempo real mediante el sistema operativo FreeRTOS incorporado en la placa STM32-Blue Pill F103RC8T6. Referente a la comunicación entre las placas, se logró una comunicación bidireccional entre las placas; teniendo como intermediario a la ESP32, logrando el envío de información entre la STM32 y la Raspberry Pi4, así como la ejecución de comandos específicos desde la Raspberry Pi4, viéndose reflejados en la STM32 y por consiguiente en las acciones mostradas en el tractor.

Se alcanzó la implementación de elementos gráficos en una interfaz, para la visualización de los datos obtenidos de los algoritmos implementados en la STM32, así como su visualización en tiempo real, permitiendo un monitoreo fluido de los mismo, permitiendo la toma de acciones al momento.

Se consiguió la implementación de un algoritmo de visión computacional a través de una cámara funcionando en tiempo real, además de su función en un contexto agrícola.

Por lo anteriormente mencionado, se demuestra el funcionamiento del prototipo, implicando su

escalabilidad a un sistema más complejo, así como su prueba en un contexto real en el área agrícola. También se demuestra su utilidad e importancia de los sistemas de conducción inteligente en un contexto agrícola.

IV. CONCLUSIONES

La industria de la maquinaria agrícola es una industria en constante actualización, es por eso que diseñamos un tractor que integra múltiples sistemas distintos en una sola máquina como respuesta a la evolución de las necesidades de la industria. Primero creamos el sistema de movimiento principal entre la STM32 y los principales sensores y actuadores del tractor: moto-reductores, la pantalla LCD, la cámara de video, el potenciómetro y los LEDs; este sistema fue programado en C y en C++. Después generamos el sistema de telemetría; sistema conformado por la ESP32 y la Raspberry Pi 4, la ESP32 recopila los datos recopilados por la STM32 hacia la Raspberry Pi 4, de ahí, la Raspberry Pi envía los datos hacia la computadora para que sea visualizados mediante Grafana. Este sistema de telemetría fue programado usando los bash scripts de Python para el control de esta última placa. Se usó el lenguaje C++ para la configuración de los pines de ESP32 para que funcione en conjunto. El control principal de las tareas de los sistemas del tractor se logró usando la librería "FreeRTOS" en C++ para la STM32, dándonos un sistema operativo en tiempo real que nos permitió manejar las distintas tareas con orden y tiempo/deadlines en nuestro sistema. El chasis del tractor fue diseñado usando SolidWorks e impresión en 3D, se buscó una representación más precisa de un tractor de John Deere pero a escala. Finalmente, la integración de todos estos sistemas nos dio como resultado un tractor funcional, elegante y limpio que cumple con los nuevos requerimientos de la industria y mejoró nuestra comprensión de cómo se trabaja en proyectos de ingeniería multidisciplinarios como equipo.

REFERENCES

- [1] Admin. (2025, December 7). Principales averías en tractores agrícolas y sus soluciones. Sumirec - Especialistas En Maquinaria Y Recambios Agrícolas. <https://sumirec.com>

- [2] Pérez, M. (n.d.). Rendimiento de un tractor agrícola en función del sistema de labranza y la carga. I. Características de la tracción. <https://scielo.org>
- [3] Jarén, C. J. (2008, September 25). Riesgos comunes y genéricos del tractor y la maquinaria agrícola. Centre de Mecanització Agrària. <https://gencat.cat>
- [4] Portal, I. W. (2024, June 13). ¿Qué es el WIFI? INCOM SHOP. <https://incom.mx>
- [5] Rohde and Schwarz. (2026). Entendiendo el UART. Rohde-Schwarz Essentials. <https://rohde-schwarz.com>
- [6] MQTT - the standard for IoT messaging. (2026). <https://mqtt.org>
- [7] Mayorquin, D. (2025, 17 junio). RTOS: Conceptos básicos y líderes del mercado. Cidei. <https://cidei.net>
- [8] Sahaj Software. (2024, 25 julio). The Importance of Microcontrollers in Modern Technology. Sahaj Software. <https://sahaj.ai>
- [9] Schneider, J., y Smalley, I. (2025, 17 noviembre). Microcontroller. IBM Think. <https://ibm.com>
- [10] Schneider, J., y Smalley, I. (2025, 17 noviembre). Microcontroller. IBM. <https://ibm.com>
- [11] Susnjara, S., y Smalley, I. (2025, 17 noviembre). Real-time operating system (RTOS). IBM. <https://ibm.com>
- [12] Wolf, M. (2004). Computers as Components: Principles of Embedded Computing system design. <https://doi.org>
- [13] Yaghmour, K., Masters, J., y Ben, G. (2008). Building embedded linux systems, 2nd edition. O'Reilly y Associates, Inc. <http://acm.org>

ANEXO A

Código fuente del proyecto

[GitHub - Vista general](#)
[GitHub - Avance 1](#)
[GitHub - Avance 2](#)
[GitHub - Versión 1: Sistema del Tractor](#)
[GitHub - Versión final: Sistema del Tractor](#)

ANEXO B

Pruebas de funcionalidad del proyecto

[Drive - Pruebas de funcionalidad](#)

ANEXO C

Recursos

[Hoja de calculo - Mapa de pines](#)
[Hoja de calculo - Recursos \(Hardware y Software\)](#)

ANEXO D

Figuras

[Drive - Figuras empleadas](#)